

```

from maix import nn, camera, display, image, time, touchscreen, app
import os
from maix import gpio, pinmap

# Define the pins for the LEDs
pinmap.set_pin_function("A15", "GPIOA15")
led2 = gpio.GPIO("GPIOA15", gpio.Mode.OUT) # LED for known face
led2.value(0) # Turn off initially
pinmap.set_pin_function("A16", "GPIOA16")
led1 = gpio.GPIO("GPIOA16", gpio.Mode.OUT) # LED for unknown face
led1.value(0) # Turn off initially

# Global Variables
pressed_flag = [False, False, False]
learn_id = 0
person_lookup = {} # To store Person_ID to Human-readable Name mapping
recognizer = None # Face recognizer object

# Load the person lookup table from the file
def load_person_lookup():
    global person_lookup
    if os.path.exists("/root/person.txt"):
        with open("/root/person.txt", "r") as f:
            for line in f:
                parts = line.strip().split("=")
                if len(parts) == 2:
                    person_lookup[parts[0]] = parts[1].strip("'") # Remove quotes and
            print("Person lookup table loaded.")
    else:
        print("No lookup table found. Starting fresh.")

# Save the person lookup table to the file
def save_person_lookup():
    with open("/root/person.txt", "w") as f:

```

Fig. 3: Main.py file (author code snippet)

method is to install MaixVision on the computer and upload files from within the IDE.

- On the right side of MaixVision IDE, open Device File Manager.
- Select the drive where the file should be uploaded and press the Upload button.

Important note. Nothing should be uploaded to the `/boot` directory, as it may corrupt the operating system, requiring a reinstallation.

The system can learn and store new faces in real time. Each new face is assigned a unique ID and saved in the database. Simultaneously, the person.txt file is populated with the ID and a generic name. All saved faces can be cleared, and the file can be manually updated with actual names.

The MaixVision IDE interface is similar to MaixPy or OpenMV IDE, making it familiar for users.

To connect the MaixCam:

1. Click the Connect icon in the bottom-left of MaixVision IDE.
2. This establishes a connection via Wi-Fi LAN.
3. Once connected, the MaixCam

can be accessed from anywhere within the local network (LAN).

Boot start

Unlike other Sipeed boards, where code is placed in *boot.py* for automatic execution, MaixCam requires a different approach:

1. In MaixVision IDE, go to the bottom-left panel and select Package
2. Enter the required details: ID, name, version, developer, and description
3. Submit and install the package.

This creates a link in the `/maixapp/apps/app.info` file with the assigned name and generates a

folder named `facerecognition` inside `/maixapp/apps/`, which contains three files, including main.py (the code.py file). Fig. 3 shows the *Main.py* file (author code snippet).

Setting auto-start

To enable auto-start:

1. Open `/maixapp/auto\_start.txt`
 2. Enter: 'facerecognition'
 3. Save the file. Ensure this file contains no other entries.
 4. Disconnect the MaixCam board.
- On reboot, the code will auto-start.

Testing

To test the system, power on the device and allow the camera to capture a person entering the frame. The system then compares the detected face against the stored database. If the face is not found, the red LED lights up, indicating 'Unknown Person.' However, if the face is recognised, the green LED lights up, and the gate opens for the authorised individual. Fig. 4 shows the various face repositioning.

Deploying the device in a small office

The device can be deployed in a small office in the following ways: real-life deployment methods, data management, modifying or deleting entries, and a summary of key features.

Real-life deployment method.

Step 1: Enroll employee faces

1. Position the camera to capture the employee's face clearly.

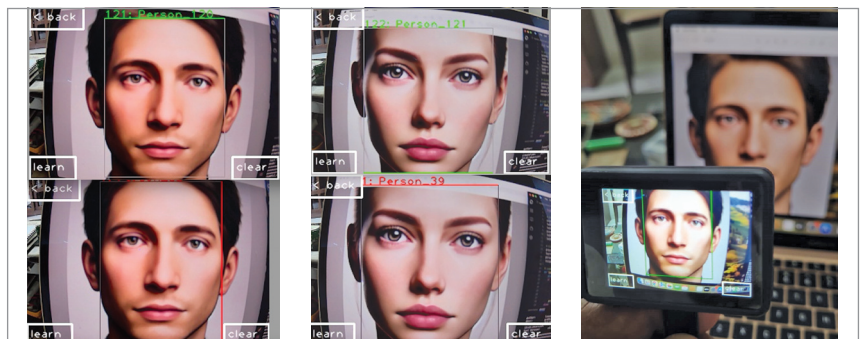


Fig. 4: Various face repositioning